Detecção de Bad Smells e Refatoração Segura

Disciplina: Testes de Software
Trabalho: Detecção de Bad Smells e Refatoração Segura
Aluno: Bernardo Souza Alvim
Matrícula: 859148
Repositório: https://github.com/alvim-puc/bad-smells-js-refactoring

1. Análise de Smells

Arquivo analisado: `src/ReportGenerator.js`, que gera relatórios CSV/HTML de itens conforme o perfil do usuário (ADMIN vê tudo; USER vê só itens com valor ≤ 500).

1.1 Método Longo

`generateReport` (linhas 11–68) concentra cabeçalho, filtro por permissão, formatação das linhas, cálculo do total e rodapé em um único método de cerca de 60 linhas. Os comentários `// --- Seção ... ---` já marcam onde estariam os métodos que deveriam existir.

Por que é problemático: o método faz coisas demais (viola a responsabilidade única) e é difícil de ler. Para os testes, só é possível exercitá-lo inteiro: a regra de visibilidade, o cálculo do total e a formatação não podem ser verificados separadamente, e uma falha em qualquer parte exige investigar o método todo.

1.2 Muitos `if/else` aninhados (complexidade alta)

O corpo do `for` aninha até quatro níveis (`for` → `if role` → `if value` → `if reportType`), e `reportType === 'CSV'` / `'HTML'` é testado em três pontos (cabeçalho, corpo, rodapé).

Por que é problemático: cada novo formato ou perfil obriga a editar vários pontos do método e multiplica os caminhos a cobrir por testes (perfil × formato × faixa de valor). A leitura exige manter vários níveis de condição na cabeça.

1.3 Código Duplicado

A linha CSV, a linha HTML e o `total += item.value` aparecem repetidos nos ramos ADMIN e USER (o `total +=` aparece quatro vezes).

```
// ramo ADMIN
report += `${item.id},${item.name},${item.value},${user.name}\n`;
total += item.value;
// ramo USER (idêntico)
report += `${item.id},${item.name},${item.value},${user.name}\n`;
total += item.value;
```

Por que é problemático: uma mudança de formato precisa ser feita em vários lugares, e é fácil esquecer um deixando os ramos inconsistentes (violação do DRY). Os testes precisam cobrir a mesma lógica em cada cópia.

1.4 Outros smells identificados

- **Números e strings mágicos:** `1000`, `500`, `'ADMIN'`, `'USER'`, `'CSV'`, `'HTML'` soltos no código, sem nome que explique seu significado de negócio.
- **Atributo não usado:** `this.db` é atribuído no construtor e nunca lido (código morto).
- **Efeito colateral:** `item.priority = true` altera o objeto recebido por parâmetro.

2. Relatório da Ferramenta

Foram instalados `eslint` (8.57.1) e `eslint-plugin-sonarjs` (0.21.0), com `plugin:sonarjs/recommended` e o limite de `sonarjs/cognitive-complexity` em 5 (mais rígido que os 15 do modelo do enunciado). Saída de `npx eslint src/` antes da refatoração:

```
src/ReportGenerator.js
  11:3   error  Refactor this function to reduce its Cognitive Complexity from 27 to the 5 allowed
sonarjs/cognitive-complexity
  43:14  error  Merge this if statement with the nested one
sonarjs/no-collapsible-if

✖ 2 problems (2 errors, 0 warnings)
```

Comparação com a análise manual:

Smell manual	Detectado?	Regra
Método Longo	Indireto	(aparece na complexidade do mesmo método)
Muitos `if/else`	**Sim**	`sonarjs/cognitive-complexity` (27 > 5)
Números mágicos	Não	nenhuma regra ativa
Código duplicado	Não	`no-identical-functions` e `no-duplicated-branches` não dispararam: a duplicação são linhas repetidas em ramos diferentes, não funções ou ramos inteiros idênticos
Atributo `db` não usado	Não	`no-unused-vars` não cobre propriedades de instância

O que a ferramenta acrescentou: ela quantificou a complexidade (27, mais de cinco vezes o limite), um número que a leitura manual só sugeria como "muitos `if/else`", e achou o `if` colapsável da linha 43, que passou despercebido na leitura. Em contrapartida, a análise manual encontrou smells que as regras ativas não modelam (valores mágicos, duplicação por linha, atributo morto). As duas abordagens se complementam: a ferramenta é objetiva e repetível; a leitura capta o significado de negócio.

3. Processo de Refatoração

Smell mais crítico: a complexidade cognitiva de `generateReport` (27), que também causava a duplicação e o tamanho do método.

Antes

```
for (const item of items) {
  if (user.role === 'ADMIN') {
    if (item.value > 1000) { item.priority = true; }
    if (reportType === 'CSV') {
      report += `${item.id},${item.name},${item.value},${user.name}\n`;
      total += item.value;
    } else if (reportType === 'HTML') {
      const style = item.priority ? 'style="font-weight:bold;"' : '';
      report += `<tr ${style}><td>${item.id}</td>...</tr>\n`;
      total += item.value;
    }
  } else if (user.role === 'USER') {
    if (item.value <= 500) {
      if (reportType === 'CSV') { /* mesma linha CSV e total +=  */ }
      else if (reportType === 'HTML') { /* mesma linha HTML e total += */ }
    }
  }
}
```

Depois (`src/ReportGenerator.refactored.js`)

```
const VISIBILITY_RULES = Object.freeze({
  [ROLES.ADMIN]: () => true,
  [ROLES.USER]: (item) => item.value <= USER_MAX_VALUE,
});

const FORMATTERS = Object.freeze({
  [REPORT_TYPES.CSV]: new CsvFormatter(),
  [REPORT_TYPES.HTML]: new HtmlFormatter(),
});

generateReport(reportType, user, items) {
  const formatter = FORMATTERS[reportType];
  if (!formatter) { return ''; }

  const visibleItems = this.filterVisibleItems(user, items);
  const total = this.calculateTotal(visibleItems);

  return [
    formatter.header(user),
    this.buildBody(formatter, user, visibleItems),
    formatter.footer(total),
  ].join('').trim();
}
```

Técnicas aplicadas

- **Extract Method:** `generateReport` virou orquestrador; filtro, total, corpo e destaque foram para `filterVisibleItems`, `calculateTotal`, `buildBody` e `isPriority`.
- **Replace Conditional with Polymorphism (Strategy):** os `if (reportType === ...)` repetidos deram lugar a `CsvFormatter` e `HtmlFormatter`, com `header`, `row` e `footer`, escolhidos uma única vez em `FORMATTERS`. Um novo formato passa a ser uma nova classe, sem editar o fluxo existente.
- **Decompose Conditional:** a regra "quem vê o quê" saiu do laço e virou a tabela `VISIBILITY_RULES`.
- **Eliminação de duplicação:** filtragem e formatação foram separadas, então cada linha CSV/HTML é gerada em um só lugar e o total é calculado uma vez, com `reduce`.
- **Replace Magic Number with Symbolic Constant e Remove Dead Code:** `ROLES`, `REPORT_TYPES`, `PRIORITY_THRESHOLD` e `USER_MAX_VALUE` nomeiam os valores; o atributo `db` foi removido e o objeto de entrada deixou de ser alterado.

Correção do `<tr >`

O teste de HTML para Admin falhava no original porque `<tr ${style}>` deixava um espaço quando não havia destaque (`<tr >`, e o teste espera `<tr>`). Na versão refatorada, o espaço passou a fazer parte do atributo:

```
const attrs = highlighted ? ' style="font-weight:bold;"' : '';
`<tr${attrs}><td>...`   // "<tr>" ou '<tr style="font-weight:bold;">'
```

Validação

Verificação	Resultado
`npx eslint src/ReportGenerator.refactored.js`	0 erros (original: 2)
Testes do arquivo refatorado	5 de 5 passam
Testes do arquivo original	4 de 5 passam; a falha é o `<tr >`, mantido porque o enunciado proíbe alterar o original
Comparação original × refatorado (3 formatos × 3 perfis × lista com itens/vazia)	saídas idênticas, exceto o `<tr >` corrigido

O teste refatorado (`tests/ReportGenerator.refactored.test.js`) é cópia do original com apenas o `import` alterado.

4. Conclusão

A suíte de testes foi a rede de segurança que tornou possível reescrever a estrutura inteira do gerador (de um método com condicionais aninhadas para formatadores e regras separados) sem medo de mudar o comportamento. Cada execução de `npm test` confirmava que a saída continuava a esperada, e o próprio teste que falhava no código original apontou um defeito real (`<tr >`), corrigido na versão nova. Como os testes verificam a saída e não a implementação, a mudança de design não exigiu alterá-los. Uma limitação: a suíte cobre poucos cenários (não testa tipo de relatório ou perfil desconhecido, por exemplo), por isso também comparei as duas versões em uma matriz maior de entradas.

Reduzir os smells melhorou a qualidade geral: a complexidade do código analisado caiu de 27 para dentro do limite de 5 por método e o ESLint passou de 2 erros para nenhum; a duplicação foi eliminada; e o código ficou mais fácil de ler e de estender, já que um novo formato ou perfil agora é uma adição isolada. A experiência também mostrou que análise estática e leitura manual se complementam: a ferramenta mediu a complexidade, e a leitura achou o que ela não detecta.